# Spherical splits for decision trees and random forests

Mathias Valla[a,*]

[a] *Chaire DIALog, Institut Louis Bachelier, Paris, France*

## Abstract

Classical decision trees recursively split the covariate space with coordinate thresholds. This rectangular geometry is simple and effective, but it can be inefficient when a label or response is controlled by membership in a localized region of feature space. We study a small modification of tree induction in which a node sends observations left or right according to whether they fall inside or outside a learned hypersphere. The rule keeps the usual recursive tree architecture while replacing some threshold questions by distance-to-center questions. We describe the split search, show that distant sphere centers recover linear frontiers as a local limiting case, and note that CART cost-complexity pruning applies without changing the pruning criterion. Exploratory benchmarks on complete PMLB datasets compare axis-aligned, oblique, and spherical trees and forests. Spherical random forests obtain the best mean balanced accuracy among the forest methods in classification, while ordinary and oblique forests remain stronger in regression and single spherical trees help mainly in low-dimensional nonlinear cases. Spherical splits are therefore best viewed as a complementary local geometry for interpretable tree-based pattern recognition.

*Keywords:* decision trees, random forests, spherical splits, recursive partitioning, pattern recognition

## 1. Introduction

Decision trees are attractive because they are nonparametric, interpretable, and easy to ensemble. Their usual form, however, fixes a strong geometry. At each internal node, CART-style trees select a feature index $j$ and a threshold $a$, then split observations according to $x_j \leq a$ or $x_j > a$ [1]. Recursively, this produces rectangular cells.

Rectangular cells are natural when the signal is sparse and coordinate-aligned. They are less economical when the target depends on an area around a center. In classification, a class may correspond to a compact region; in regression, a response surface may change around a point, centroid, or prototype. An axis-aligned tree can approximate such a region, but it may need many cuts. A spherical split can isolate the same region directly.

This geometry is also different from oblique splitting. An oblique rule asks whether a weighted linear combination of covariates exceeds a threshold [3]. This is flexible, but once several variables are involved the resulting score can be difficult to interpret. A spherical rule asks whether an observation lies within radius $r$ of a center $c$. When the active covariates define a meaningful metric space, this can be a natural local-neighborhood statement. For instance, with projected latitude and longitude, a split describes membership in a circular region around a geographical point. This interpretation depends on scaling and on the chosen metric, but it can be more direct than an abstract linear score.

* Corresponding author.
*Email address:* mathias.valla@gmail.com (Mathias Valla)

## 2. Spherical splitting rules

Let $S \subseteq \{1, \ldots, p\}$ be the set of features used at a node. A spherical split is defined by a center $c \in \mathbb{R}^{|S|}$ and a radius $r \geq 0$:

$$\|x_S - c\|_2^2 \leq r^2.$$

Observations satisfying this inequality are sent to one child and the remaining observations to the other. The pair $(c, r)$ is selected to maximize the same impurity decrease used in ordinary tree induction, such as Gini decrease for classification or variance reduction for regression.

In the implementation studied here, candidate centers are generated from a finite mixture of rules: global centers, target-conditional centers, observed points, pairwise midpoints, local Gaussian perturbations, and farther radial perturbations. For each candidate center, candidate radii are induced by the Euclidean distances from the center to node samples. This preserves the usual finite split search structure: the geometry changes, but the recursive optimization problem remains a sequence of impurity comparisons.

Figure 1 compares three split geometries on two-dimensional examples. The learned spherical rule at each internal node is a circle; terminal regions arise from recursive inside/outside decisions.

## 3. Geometry, pruning, and computation

A first useful case is radial or local structure. If the target changes when $x$ enters a ball around $c$, then a single spherical split can represent that event. A rectangular tree must approximate the ball by coordinate cuts. This is an approximation-

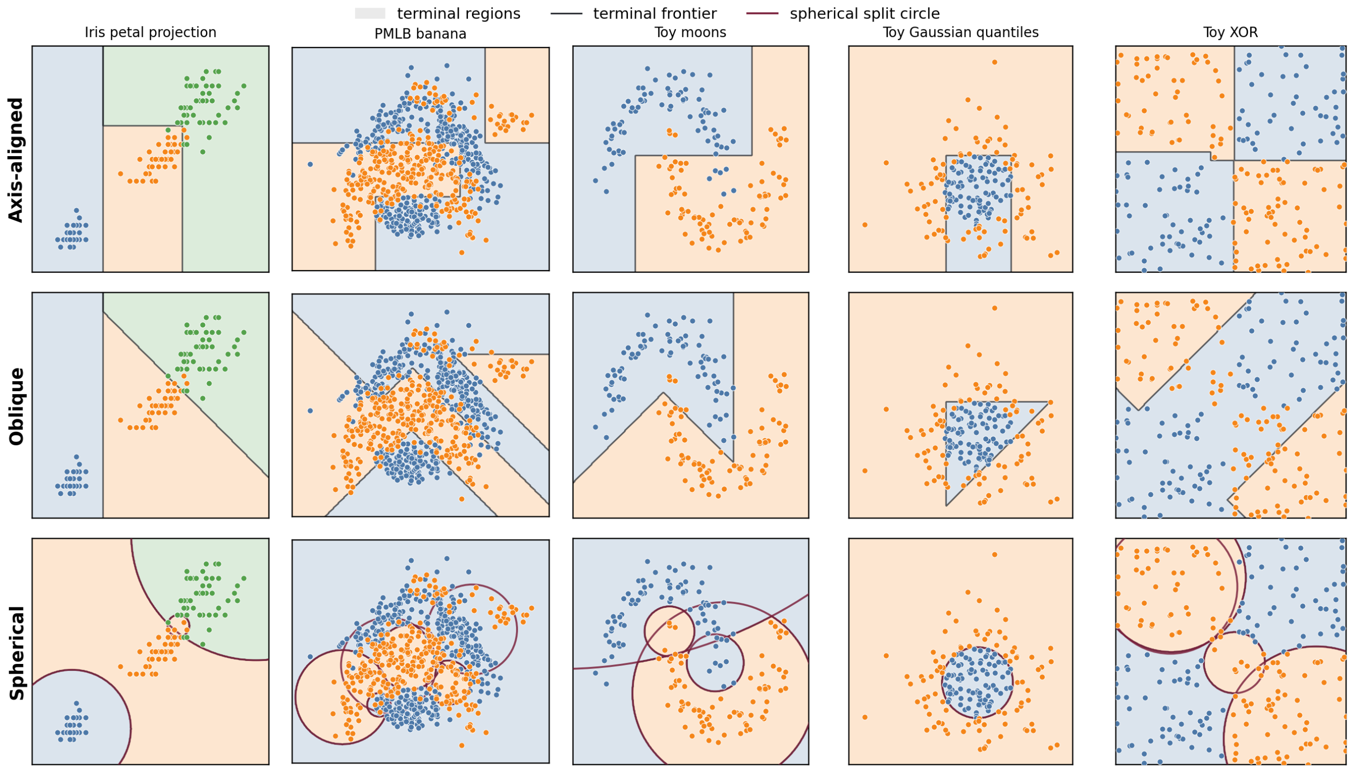


Figure 1: Two-dimensional decision regions learned by shallow axis-aligned, oblique, and spherical trees. Each row is a dataset and each column is a splitting geometry. Black curves are terminal decision frontiers; burgundy circles in the spherical panels are internal split boundaries. The panels illustrate when spherical geometry is economical, for example in localized or curved class structures, and show that greedy induction can still be imperfect on interactions such as XOR.

economy argument rather than a claim that spherical trees are universally more expressive.

A second useful case is the limiting connection to linear splits. Let $u \in \mathbb{R}^p$ satisfy $\|u\|_2 = 1$, fix $t \in \mathbb{R}$, let $R$ be large enough that $R^2 - 2Rt \geq 0$, set $c = Ru$, and choose

$$r_R^2 = R^2 - 2Rt.$$

Then

$$\|x - Ru\|_2^2 \leq r_R^2 \iff \|x\|_2^2 - 2Ru^\top x + R^2 \leq R^2 - 2Rt$$
$$\iff u^\top x \geq t + \frac{\|x\|_2^2}{2R}.$$

On any bounded region $\|x\|_2 \leq B$, the final term is at most $B^2/(2R)$. Thus the spherical split converges locally to the half-space $u^\top x \geq t$ as the center is moved far away. Axis-aligned splits are recovered by taking $u$ equal to a coordinate vector. In finite samples this is only a local approximation, but it motivates center-generation schemes that include distant candidates.

The induced cells also have a simple algebraic form. A leaf at depth $d$ is an intersection of at most $d$ sets, each of which is either a ball or the complement of a ball. Spherical-tree leaves are therefore semi-algebraic cells defined by quadratic inequalities. In two dimensions their boundaries are made of circular arcs; in higher dimensions they are made of hyperspherical patches. This contrasts with axis-aligned boxes and oblique polyhedra, and it gives a compact way to express annuli or shells by combining outside and inside sphere decisions.

The split can also be read as a restricted quadratic rule:

$$\|x - c\|_2^2 \leq r^2 \quad \iff \quad x^\top x - 2c^\top x + \|c\|_2^2 - r^2 \leq 0.$$

Equivalently, it is an oblique split in the lifted representation $z(x) = (x, \|x\|_2^2)$, with the coefficient of $\|x\|_2^2$ fixed to be positive. This places spherical trees between oblique trees and general quadratic trees: they add isotropic curvature without introducing the full parameter count, and loss of interpretability, of arbitrary quadratic forms.

Cost-complexity pruning adapts directly. CART pruning selects subtrees by minimizing

$$R_\alpha(T) = R(T) + \alpha|\widetilde{T}|,$$

where $R(T)$ is the empirical leaf risk and $|\widetilde{T}|$ is the number of terminal nodes [1]. Once a spherical tree has been grown, this criterion depends only on topology, leaf predictions, leaf risks, and leaf count. It does not depend on whether internal nodes were created by coordinate, oblique, or spherical rules. The usual weakest-link pruning sequence and validation choice of $\alpha$ can therefore be used without changing the statistical pruning criterion.

The current implementation is a research prototype rather than an optimized solver. At a node with $n_m$ samples, $s$ active features, and $C$ candidate centers, the basic distance computation costs $O(Cn_m s)$. If all data-induced radii are considered, sorting the distances for each center adds $O(Cn_m \log n_m)$

work, after which cumulative impurity updates scan the ordered candidates. These costs explain slower fits, but they are not a lower bound for the model class. Squared norms can be reused in $\|x-c\|_2^2 = \|x\|_2^2 - 2x^\top c + \|c\|_2^2$; many centers can be evaluated by matrix multiplication; center evaluations are parallel at a node; approximate variants can use quantiles or histograms; and forests add another level of parallelism.

## 4. Experiments

We implemented spherical trees and spherical random forests in `treeple`. The code and scripts used to reproduce the results are available at https://github.com/MathiasValla/spherical-decision-trees. We compared the spherical estimators with axis-aligned CART and random forests, as well as oblique trees and forests. The quantitative benchmark used complete PMLB datasets that ran without errors [4]; the toy datasets in Figure 1 are qualitative illustrations only. Features were imputed and standardized. The benchmark used three cross-validation folds, 50 trees for forests, 500 spherical center candidates per node, all data-induced radii for each candidate center, validation-selected pruning for single trees, and unpruned forests. Large datasets were capped at 1000 sampled observations. The final comparison contained 249 complete datasets and 38 skipped datasets or fetch/model errors. The comparison is descriptive: it is intended to characterize regimes and failure modes, not to prove dominance.

The salient empirical result is the classification performance of spherical forests. In this exploratory benchmark, spherical random forests obtain the best mean classification score among the compared methods, outperforming both ordinary and oblique random forests, although at much larger computational cost. This advantage is not observed generally for regression, where ordinary and oblique random forests remain stronger. Nor is it observed on average for single trees. The evidence therefore points to a specific use case: spherical split candidates can be valuable when their high-variance behavior is stabilized by ensembling, especially in classification problems with localized, radial, or cluster-like structure. Dataset-level paired tests and larger computational studies are left for future work.

Figure 2 shows why the result should be read as a statistical signal rather than as a practical speed claim. Spherical forests are the strongest classification model on mean score, but they are currently orders of magnitude slower than ordinary forests. The regression panel also clarifies a negative result: spherical geometry alone does not improve the forest benchmark when the target is continuous.

## 5. Discussion and limitations

Spherical split search is much more expensive than scanning coordinate thresholds. This matters especially in forests, where the cost is repeated across many trees and internal nodes. The method also depends on the metric. In our experiments, predictors were standardized before fitting. Without scaling, a spherical split is dominated by high-variance coordinates. In high-dimensional spaces, Euclidean distances can become less informative, and far-away centers may make splits nearly linear without solving the underlying greedy-search problem.

The XOR example is instructive. In principle, two far-away circles can locally approximate the two axis-aligned cuts needed for XOR. A greedy impurity criterion, however, may reject those cuts at the root because each individual axis split has little immediate impurity gain. This does not contradict the linear-limit argument. It shows that spherical trees inherit the myopia of ordinary greedy tree induction. A depth-two lookahead or delayed-gain criterion would be needed to systematically recover such interactions. For single trees, the more favorable low-dimensional cases are nonlinear settings with compact or curved decision regions, such as moon-shaped classes, the PMLB banana classification problem, or radial Gaussian quantiles.

## 6. Conclusion

Spherical splits provide a simple geometric extension of recursive partitioning. They are most appealing when the target is expected to depend on membership in an area of the covariate space, especially around a center, prototype, cluster, or spatial location. Because distant centers recover linear frontiers locally, the same mechanism can also approximate hyperplane splits. The main empirical finding is that spherical forests improved mean classification performance in the benchmark studied here. This improvement is not general for regression, and single spherical trees are competitive mainly in specific low-dimensional nonlinear regimes.

The natural next step is a hybrid tree that chooses among axis-aligned, oblique, and spherical split candidates at each node, possibly with a small lookahead window. Such a model would keep the interpretability of recursive partitioning while allowing the geometry of each decision to adapt locally.

## Data and code availability

The PMLB datasets used in the experiments are publicly available [4]. An open-source implementation of spherical trees, together with scripts for reproducing the results presented in this paper, is available in the `treeple`-based repository at https://github.com/MathiasValla/spherical-decision-trees. The repository includes the file `REPRODUCING_RESULTS.md`, which documents how to retrieve the stored result tables and regenerate all figures and results reported in this article.


## Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Table 1: Mean benchmark performance and fit time over complete datasets. Primary score is balanced accuracy for classification and $R^2$ for regression.

| Task | Model | Datasets | Score | Fit time (s) |
|---|---|---|---|---|
| Classification | Spherical random forest | 132 | 0.765 | 9.850 |
| Classification | Random forest | 132 | 0.738 | 0.137 |
| Classification | Oblique random forest | 132 | 0.737 | 0.154 |
| Classification | CART | 132 | 0.722 | 0.074 |
| Classification | Oblique tree | 132 | 0.706 | 0.136 |
| Classification | Spherical tree | 132 | 0.688 | 4.132 |
| Regression | Random forest | 117 | 0.717 | 0.384 |
| Regression | Oblique random forest | 117 | 0.700 | 0.434 |
| Regression | Spherical random forest | 117 | 0.665 | 20.533 |
| Regression | CART | 117 | 0.521 | 0.078 |
| Regression | Oblique tree | 117 | 0.391 | 0.093 |
| Regression | Spherical tree | 117 | 0.262 | 7.516 |

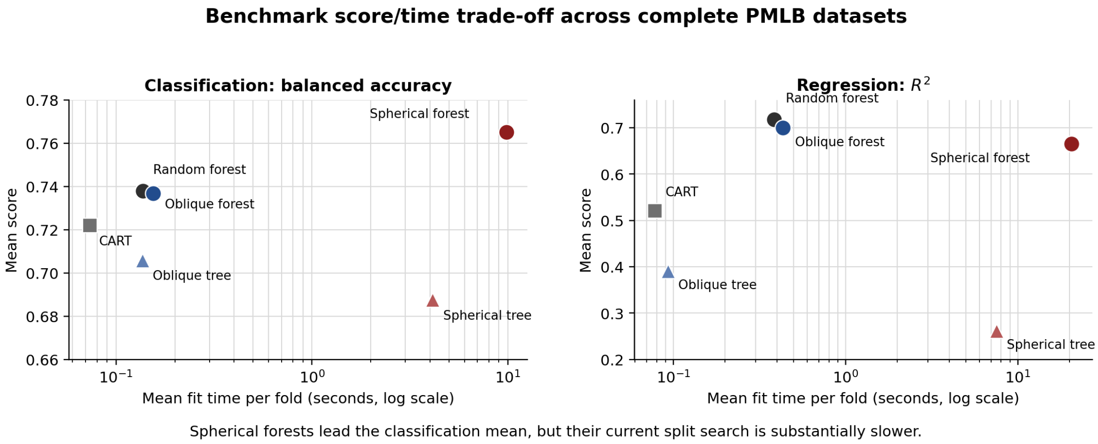


Figure 2: Mean score and mean fit time over complete PMLB datasets. Scores are balanced accuracy for classification and $R^2$ for regression. The figure highlights the main empirical trade-off: spherical forests obtain the best mean classification score among the compared forest methods, but the current spherical split search is substantially slower; in regression, ordinary and oblique forests remain both stronger and faster.

## Funding

This work was conducted within the Research Chair DIALog under the aegis of the Risk Foundation, a joint initiative by Université Paris-Dauphine PSL and CNP Assurances.

## Acknowledgments

Work(s) conducted within the Research Chair DIALog under the aegis of the Risk Foundation, a joint initiative by Université Paris-Dauphine PSL and CNP Assurances.

## Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used OpenAI's Codex and ChatGPT to assist with language editing, formatting, code development, and document iteration. The author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

## References

[1] Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. *Classification and Regression Trees*. Wadsworth, 1984.
[2] Breiman, L. Random forests. *Machine Learning*, 45:5–32, 2001.
[3] Murthy, S. K., Kasif, S., and Salzberg, S. A system for induction of oblique decision trees. *Journal of Artificial Intelligence Research*, 2:1–32, 1994.
[4] Olson, R. S., La Cava, W., Orzechowski, P., Urbanowicz, R. J., and Moore, J. H. PMLB: a large benchmark suite for machine learning evaluation and comparison. *BioData Mining*, 10:36, 2017.
[5] Pedregosa, F. et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
[6] Valla, M. and NeuroData. Spherical decision trees: `treeple`-based implementation and reproduction scripts. https://github.com/MathiasValla/spherical-decision-trees, 2026.